

NTU · Data Analysis and Machine Learning with Python · Spring 2026

# WaferDX

Explainable AI-Based Wafer Map  
Defect Diagnosis and Decision Support System

---

Abhirup Sain · David Spickenheuer · Martin Werner ·  
Matti Lehmann · Staniya Thomas

June 2026

# The Problem: Wafer Defect Diagnosis

- ▶ Chips manufactured on thin circular silicon discs — **wafers** — with hundreds of chips etched side-by-side
- ▶ After probe test, every die result is stored as a pixel: **0 = background, 1 = pass, 2 = fail**
- ▶ The **spatial pattern** of failures — not the count — identifies the responsible process step

## Ring around edge

→ RTP thermal non-uniformity

## Linear streak

→ Robot arm damage / CMP

## Why speed matters

- ▶ Modern fabs produce **thousands of wafers per day** — manual review impossible
- ▶ Every undetected fault processes **more wafers** under failure conditions
- ▶ Getting it wrong costs **millions in materials and machine time**

WM-811K dataset: **811,457** total wafer maps · **only 12,822** carry a defect label

## WM-811K Dataset: 8 Defect Classes + None

### Center

Failure cluster at wafer centre → CVD deposition drift / spin-coat blockage

### Donut

Ring-shaped failure band → non-uniform spin coating / CMP pressure ring

### Edge-Loc

Localised arc of failures on edge → edge-bead removal issue

### Edge-Ring

Full ring of failures at wafer perimeter → RTP thermal non-uniformity

### Loc

Localised cluster anywhere on wafer → particle contamination

### Random

Scattered failures with no spatial pattern → random contamination events

### Scratch

Linear streak of failures → robot end-effector damage / CMP over-polishing

### Near-full

Almost all dies failing → catastrophic process excursion

**Dataset sparsity:** 811,457 total maps · only 12,822 labelled defects ( 1.6% ) · **None** class = clean wafer · three-valued pixel encoding {0, 1, 2} specific to probe test output format

# Decision-Making Challenges Faced by Engineers

## Visual ambiguity

- ▶ **Edge-Loc** and **Edge-Ring** both cluster failures near the wafer perimeter - hard to tell apart
- ▶ Several defect types share visual features at **64×64 resolution** - difficult even for experts

## Scale makes manual review impossible

- ▶ Modern fabs: **thousands of wafers per day**
- ▶ No bandwidth to hand-examine every map
- ▶ Existing tools return a category — **not** which spatial region drove it

## Co-occurring defects

- ▶ Contamination event → random scatter **and** process drift → edge failures **simultaneously**
- ▶ WM-811K provides **only single-defect labels** — real world is multi-defect

## Existing tools: what but not where

- ▶ Single-label classifiers don't handle **co-occurring** defect types
- ▶ Grad-CAM only produces **one blended heatmap**, not per-class spatial breakdown
- ▶ All existing tools are **passive classifiers**

# Related Design Approaches: Three Historical Camps

## 1 · Rule-Based Template Matching

- ✓ Transparent · Fast
- ✗ Cannot adapt to unseen morphologies
- ✗ No concept of multi-defect scenarios
- ✗ Brittle when defect looks slightly different from template

## 2 · Classical ML (SVM / RF)

- ✓ More flexible than templates ✓ Works on hand-crafted features
- ✗ Requires domain expertise to engineer features ✗ Brittle when fab conditions change

## 3 · Deep CNNs

- ✓ Impressive accuracy on WM-811K ✓ Multi-label variants handle co-occurrence
- ✗ Black box — score but no reason ✗ Grad-CAM = single blended map, not per-class ✗ All are passive classifiers, not interactive tools

**WaferDX differentiates by:** Per-class spatial activation maps as **primary output** (not post-hoc) · Multi-label capability **baked into training data design** · Interactive **draw mode** for hypothesis testing

# System Architecture: Two-Stage Pipeline

## Stage 1 — Segmentation Model (U-Net-inspired · 3.21 M parameters)

- ▶ Encoder: 4 conv blocks at filter depths **32, 64, 128, 256** + MaxPooling
- ▶ Bottleneck: GlobalAveragePooling → two Dense layers → reshape + upsample
- ▶ Skip connections let decoder recover spatial detail lost in bottleneck
- ▶ Output: **8-channel map at 96×96 resolution** — one channel per defect class
- ▶ Acts as **spatial analyst**: translates raw wafer map into 8 spatial overlay maps

## Streamlit UI (app\_best\_wafer\_cnn.py)

- ▶ Loads **best\_wafer\_cnn.keras** trained model
- ▶ **Upload mode**: engineer drags in a wafer map PNG/JPG/JPEG
- ▶ **Draw mode**: freehand canvas sketch for hypothesis testing
- ▶ Results: **3×3 grid of confidence cards** — defect name + percentage + colour-coded bar

Cyan/blue: confidence above 70% · Grey: below 30% · Gradient in between

# System Architecture: Two-Stage Pipeline

## Stage 2 — Classification Model (CNN + Spatial Attention · 2.11 M parameters)

- ▶ Input: 8-channel segmentation output
- ▶ 4 conv blocks each followed by custom **SpatialAttention** layer (CBAM-inspired)
- ▶ SpatialAttention: avg + max pool across channels  $\rightarrow 7 \times 7$  conv  $\rightarrow$  soft mask
- ▶ GlobalAveragePooling  $\rightarrow 2 \times$  Dense(512) + BatchNorm + Dropout
- ▶ Output: **8 sigmoid probabilities** — one per defect class

## Total Parameters: 5.68 M

Segmentation: 3.21 M + Classification: 2.11 M

Comparison baseline: `app_direct_classifier_cnn.py` — same UI experience wired to direct CNN classifier (simpler, no segmentation stage)

## System Boundaries

Works with: 2D PNG/JPG/JPEG wafer maps + freehand drawings

Does NOT: write to databases, trigger alarms, flag wafers for scrap

**Role is entirely advisory — engineer decides**

## Usage Workflow: 4-Step Pipeline

**1 Input** Upload PNG/JPG/JPEG wafer map OR draw a freehand pattern on interactive canvas



**2 Preprocess** Convert to grayscale → normalise pixel values to  $\{0, 1, 2\}$  by rounding pixel/127.5  
→ resize to  $64 \times 64$  with padding to preserve aspect ratio



**3 Inference** Image → Segmentation model → 8 spatial activation maps at  $96 \times 96$  → Classification model → 8 sigmoid confidence scores



**4 Display Results** Confidence cards in  $3 \times 3$  grid · Bars cyan/blue above 70%, grey below 30%, gradient in between · Engineer inspects per-class activation maps · Engineer decides what to do



# Design Decision 1: Two-Stage vs. Direct End-to-End Classification

## Two-Stage (chosen)

- Segmentation output = **interpretable intermediate representation**
- When Edge-Ring activation lights up around wafer periphery, you can **see why** the model calls it Edge-Ring
- Multi-label capability baked into data design, not architectural add-on

## Direct End-to-End (baseline)

- Simpler to train and deploy
- Single end-to-end optimisation problem
- **Whenever it got something wrong**, it was genuinely hard to understand why
- Grad-CAM patches this partially — rough single blended heatmap, not per-class breakdown

**Trade-off:** Two-stage requires more upfront work — segmentation must stabilise before classifier pre-training starts. Pipeline is more complex to debug when both components are unfrozen during fine-tuning. Managed with **ModelCheckpoint** + **EarlyStopping** (patience 10–15 per phase).

## Design Decisions 2 & 3: Spatial Attention + Synthetic Multi-Defect Data

### Spatial Attention in Every Conv Block

Why:

- Scratch defect may cover **less than 5%** of the image — treating all locations equally wastes capacity
- CBAM-inspired: pool features across channels  $\rightarrow$   $7 \times 7$  conv  $\rightarrow$  soft spatial mask  $\rightarrow$  multiply back onto feature maps

Alternative considered:

- Channel attention (SE-Net) — focuses on **which feature channels** matter, not **where**; wrong emphasis for a geometry-defined task.

### Synthetic Multi-Defect Sample Generation

Why:

- WM-811K **only has single-defect labels**. Real wafers can have co-occurring defects from simultaneous process problems.
- Method: element-wise **maximum blending**
- No additional human annotation required

Trade-off:

- Element-wise max is a simplification
- Hardest cases: **Donut + Near-full** (both cluster at wafer edge  $\rightarrow$  merged activation blobs)

## Design Decision 4: Three-Phase Training Curriculum

### 1. Train Segmentation Model Alone

- ▶ 100 epochs, 64 steps/epoch, batch size 16
- ▶ Loss: 0.140  $\rightarrow$   $\approx$  **0.017**
- ▶ Sharp drop first 5 epochs  $\rightarrow$  slow decline
- ▶ Smooth convergence

### 2. Freeze Seg, Pre-Train Classifier Head

- ▶ 32 epochs - segmentation frozen
- ▶ AUC: 0.65  $\rightarrow$  0.86
- ▶ Loss: 0.82  $\rightarrow$  0.30
- ▶ Refined 20-epoch run: train AUC 0.93, **val AUC 0.94** at best epoch 19

### 3. End-to-End Fine-Tuning

- ▶ 50 epochs · batch size 64 · both models unfrozen
- ▶ Train AUC: 0.91  $\rightarrow$  0.95
- ▶ **Val AUC peaks above 0.96** · best epoch 40
- ▶ Train loss: 0.27  $\rightarrow$  0.20
- ▶ Validation more volatile

**Why curriculum training?** End-to-end from scratch: classifier received random-noise segmentation outputs  $\rightarrow$  erratic gradient flow. Training each stage to convergence before connecting reduces gradient interference. Broadly applicable pattern for any multi-stage deep learning pipeline.

# AI Role and System Autonomy

## The AI's Two Roles

**Segmentation model = Analyst**

- From raw wafer map → 8 spatial overlay maps
- Translates greyscale image into **where each defect type appears**

**Classification model = Interpreter**

- Reads spatial maps → outputs **calibrated probabilities** per class
- Converts spatial evidence into 8 sigmoid confidence scores

## Level of Automation

- Preprocessing, segmentation, classification: fully automatic

**System does NOT:**

- Flag a wafer for scrap
- Raise a process alarm
- Notify a supervisor
- Update any external database

**Draw mode** turns the system into an active exploration tool

# Evaluation Setup and Results: Training Phases

## Evaluation Goals

- ▶ Did segmentation model learn to **localise defect regions** (or just produce plausible noise)?
- ▶ Did classification model reach a **practically useful AUC**?
- ▶ Does two-stage pipeline produce **better/more interpretable** results than direct classifier?

## Phase 2 Extended — 50 Epochs

- ▶ Training AUC: 0.91  $\rightarrow$  0.96 by epoch 50
- ▶ Validation AUC: volatile 0.92–**0.98**, best at epoch 47 (ModelCheckpoint saved)
- ▶ Volatility: segmentation backbone gets gradient updates  $\rightarrow$  feature space shifting under classifier
- ▶ Train loss: 0.27  $\rightarrow$  0.175, Val loss minimum 0.15

## Seg. Model Convergence

MSE loss 0.140  $\rightarrow$  **0.017** over 100 epochs

## Phase 2 Best Checkpoint

Val AUC **0.94** at epoch 19/20

## Phase 3 Best Checkpoint

Val AUC **above 0.96**, best epoch 40/50

## Segmentation Model: Visual Output Before vs. After Training

### Before Training (epoch 0)

- ▶ Model outputs **undifferentiated noise** across all 8 channels
- ▶ No spatial preference whatsoever
- ▶ All channels look identical — structureless activation blobs

### After 100 Epochs of Training

- ▶ **Center** channel: strong activation near wafer middle
- ▶ **Edge-Ring** channel: circular activation at wafer periphery
- ▶ **Scratch** channel: elevated activation along linear failure streak
- ▶ **Unrelated channels** (e.g., Center, Loc, Near-full for an Edge-Ring wafer): stay near zero

## Model Comparison: 4 Architectures on 160 Test Images

| Class     | Best CNNn(no aug) | Best CNNn(+ aug) | Fullstack | Seg → CLF |
|-----------|-------------------|------------------|-----------|-----------|
| Center    | 0%                | 0%               | 0%        | 95%       |
| Donut     | 65%               | 10%              | 0%        | 0%        |
| Edge-Loc  | 30%               | 85%              | —         | 0%        |
| Edge-Ring | 0%                | 0%               | 0%        | 0%        |
| Loc       | 0%                | 0%               | 0%        | 0%        |
| Near-full | 5%                | 15%              | 100%      | 0%        |
| Random    | 5%                | 0%               | 35%       | 0%        |
| Scratch   | 0%                | 0%               | 0%        | 0%        |
| Overall   | 13.1%             | 13.8%            | 17.5%     | 11.9%     |

Random chance on 8-class balanced test = 12.5%. All models near random on most classes. Test images: 20 samples per class × 8 classes = 160 PNG images from WM-811K via generate\_images.py (fixed seed, {0,1,2} → {0,128,255} pixel scaling for visualisation only).

## Critical Failure Cases and Unexpected Outcomes

### Per-class accuracy collapse (all models)

- ▶ Every architecture collapses to high accuracy on 1–2 dominant classes while failing on the rest
- ▶ Sign of class imbalance + class-specific shortcuts rather than general spatial representations
- ▶ No single architecture achieved balanced performance across all 8 classes

### Spatial ambiguity on overlapping synthetic defects

- ▶ Donut + Near-full: both cluster near wafer edge → segmentation model produced **merged activation blobs**
- ▶ Recognised that something was happening at the edge, but not **which** specific pattern
- ▶ Known limitation — would require real labelled multi-defect data to address properly



## Critical Failure Cases and Unexpected Outcomes

### Draw mode distribution shift

- ▶ Drawn strokes are **sharper and higher-contrast** than real wafer map greyscale gradients
- ▶ Direct classifier sometimes less confident on drawn inputs than uploaded ones
- ▶ Creates distribution mismatch between draw mode and training data

### Early Phase 3 instability + sparse labels

- ▶ Validation loss spiked in first 2–3 fine-tuning epochs before settling — differential learning rates not implemented (time constraint)
- ▶ Only 12,800 of 811,000 maps labelled — thin supervision signal likely causing gaps for rare morphologies
- ▶ Seg → CLF pipeline: classifier overfit to “Center” class (confidence 1.0 for over 90% of all inputs regardless of true class)

# What the Evaluation Cannot Tell Us

## Multi-label evaluation is entirely qualitative

- ▶ No held-out test set with genuine multi-label ground truth — WM-811K **does not provide one**
- ▶ Multi-defect evaluation = looking at outputs and judging if they “looked right”
- ▶ Useful but not rigorous

## No real-world fab validation

- ▶ WM-811K comes from one fab environment with its own process conditions, tool fleet, wafer sizes
- ▶ Model may not generalise to a different fab without recalibration
- ▶ Production deployment requires **periodic retraining** on locally collected labelled data

## Deployment risk: automation complacency

When system shows 94% confidence for Edge-Ring, real risk that engineer accepts it **without checking spatial maps** — opposite of intended behaviour. Interface shows all 8 class probabilities simultaneously to make uncertainty visible and encourage critical engagement.

# Transferable Design Insights

## Generalizable Principles

- ▶ **Spatial decomposition as explainability:**  
Localisation-then-classification = intermediate spatial maps ARE the explanations. Equally useful in medical imaging, PCB inspection
- ▶ **Synthetic multi-label composition:**  
Element-wise max compositing works wherever single-label data exists and co-occurrence is physically plausible.
- ▶ **Curriculum training for multi-stage pipelines:** Train each stage to convergence before connecting. Broadly applicable for any pipeline where one model feeds another

## Specific to WM-811K Setting

- ▶ 8-class defect taxonomy is **dataset-specific**
- ▶ Three-valued pixel encoding  $\{0, 1, 2\}$  specific to **probe test output format**
- ▶  $96 \times 96$  target resolution = balance for this dataset; different datasets may need different resolution
- ▶ Element-wise max compositing assumes **spatially non-overlapping defects** — holds for WM-811K but may not elsewhere

**Distribution shift risk:** Fab-specific training data. Model trained on one fab may not generalise to another.

# Future Extensions

## Higher Resolution Inputs

- Move from  $64 \times 64$  to  **$128 \times 128$  or  $256 \times 256$**
- Especially important for **Scratch** defects — often only a few pixels wide at current resolution
- Better spatial detail: Edge-Loc vs. Edge-Ring

## Real Multi-Label Annotations

- **Most important single improvement:** multi-defect wafer maps labelled by process engineers
- Enable rigorous multi-class F1 evaluation

## Calibrated Uncertainty

- Add **Monte Carlo Dropout** or **conformal prediction**
- Give system a principled way to say “I am not sure about this one”

## Integration with Fab Systems

- Connect to **manufacturing execution system**
- Raise **SPC alerts** directly when high-confidence defect calls are made
- Connects diagnosis and process control

# Conclusion

---

## System

5.68 M parameters, Two-stage U-Net + CNN + Spatial Attention, Streamlit UI with Upload + Draw modes

## Performance

MSE loss  $\approx 0.017$  · Val AUC peaks **0.98** at epoch 47 ·  
Fullstack: 17.5% test accuracy (best of 4)

We wanted a tool telling engineers not just **what** is wrong with a wafer, but **where**  
The system is designed to support critical engagement, not discourage it.

---

Abhirup Sain · David Spickenheuer · Martin Werner · Matti Lehmann · Staniya Thomas  
NTU · Data Analysis and Machine Learning with Python · June 2026

References: LeCun et al. 2015 · Ronneberger et al. 2015 (U-Net) · Woo et al. 2018 (CBAM) · Wu et al. 2015 · Nakazawa & Kulkarni 2018 · Selvaraju et al. 2017 (Grad-CAM) · Molnar 2022